Que mesure vraiment τ^2-bench ?

Rapport 1 — Prise en main, limites du benchmark
et premier essai sur l'effet du raisonnement

Théo Fontaine Jean-Léo Serrano

M2 MIND — MEDS (méthodologie projet), Sorbonne Université

Octobre 2026

Résumé

τ^2-bench évalue des agents conversationnels à base de LLM. L'agent dispose d'outils et doit respecter une politique métier pour traiter les demandes d'un utilisateur, lui-même simulé par un LLM. Ce premier rapport décrit le benchmark (architecture de la simulation, domaines, tâches, outils, mécanisme de notation), puis analyse ses biais et ses limites. L'analyse s'appuie sur le code et les données du dépôt officiel, figés à un commit précis, et sur nos premiers essais.

Nous montrons que le score `airline` agrège deux compétences distinctes : 24 tâches sur 50 attendent que l'agent ne modifie rien (le plus souvent, refuser une demande contraire à la politique), et 26 attendent qu'il agisse correctement. Les résultats publiés ne distinguent pas ces deux familles. La notion de succès diffère aussi selon les domaines, contrairement à ce qu'indique la documentation. Les règles métier ne sont vérifiées que par leurs effets sur la base de données, et plus d'un tiers des tâches `airline` et `retail` ont dû être corrigées. Nos essais font apparaître un faux négatif (un agent pénalisé pour un champ facultatif) et un faux positif (une violation de la politique notée comme un succès). Nous menons enfin un premier essai sur l'effet d'un budget de raisonnement : `Qwen3-14B` en local, sur les 50 tâches `airline` et 3 essais par tâche. Le budget de 4096 tokens fait passer pass[1] de 30,7 % à 32,7 % et pass[2] de 18,0 % à 21,3 %. Ces écarts vont dans le sens attendu mais ne sont pas significatifs, et un score global ne dit pas sur quelle famille de tâches porte le gain. Nous en tirons des questions de recherche et un protocole pour la suite du projet.

1 Introduction

Un LLM n'est plus seulement un générateur de texte : il appelle des outils, inspecte et modifie des bases de données, et enchaîne ces actions dans une boucle « raisonner, agir, observer » [1]. On l'appelle alors un *agent*. Évaluer un agent pose des difficultés que l'évaluation classique d'un modèle ne connaît pas :

- la sortie n'est pas une réponse unique mais une *trajectoire* (messages, appels d'outils, modifications d'état) ;
- le comportement est stochastique et dépend d'un interlocuteur lui-même stochastique ;
- une erreur a des conséquences sur le monde, par exemple un remboursement injustifié ou une annulation non demandée.

τ^2-bench [2], développé par Sierra Research, prolonge τ-bench [3]. Il simule des conversations de service client entre un agent évalué et un utilisateur simulé par un LLM, dans plusieurs domaines à fortes contraintes métier (compagnie aérienne, vente en ligne, opérateur télécom, banque). Sa principale nouveauté est le cadre *dual-control* du domaine télécom : l'utilisateur dispose lui aussi d'outils, et l'agent doit le guider pour qu'il agisse sur son propre appareil.

Objectif du projet. Il ne s'agit pas d'obtenir le meilleur score. Nous voulons comprendre *ce que le benchmark mesure réellement* : le score reflète-t-il la compétence de l'agent, ou des artefacts de la

simulation et de la notation ? Nous voulons ensuite concevoir un protocole d'évaluation rigoureux, l'appliquer à deux ou trois agents simples et en rendre compte de façon critique et traçable.

Une hypothèse à tester : l'effet du raisonnement. Les modèles récents peuvent consacrer un budget de tokens à une phase de raisonnement interne avant de répondre (*reasoning models*, *test-time compute*). Une hypothèse répandue veut que ce temps de calcul améliore nettement les agents outillés : il permettrait de vérifier les préconditions de la politique avant d'appeler un outil, de planifier les actions en plusieurs étapes et de résister à un utilisateur qui insiste. Cette hypothèse est un bon cas d'étude pour notre question de fond : pour la tester, il faut savoir ce que le score mesure, et si un écart observé dépasse le bruit du benchmark.

Contenu du rapport. Ce premier rapport décrit le benchmark (sections 2 à 4). Il rend compte de notre prise en main, dont un premier essai contrôlé de l'hypothèse ci-dessus avec `Qwen3-14B` exécuté en local (section 5). Il analyse enfin les biais et les limites du benchmark (section 6).

Traçabilité. Toutes les observations portent sur le dépôt officiel [4] figé au commit `b7ea907` (version 1.0.1). Les statistiques sont produites par `code/domain_stats.py` dans notre dépôt, et chaque simulation citée est versionnée dans `results/` avec la commande qui l'a produite.

2 Fonctionnement de τ^2-bench

2.1 Une simulation à trois acteurs

Chaque épisode met en jeu trois composants, coordonnés par un *orchestrateur* qui fait alterner les tours de parole (figure 1) :

L'agent évalué est un LLM. Son prompt système contient la *politique* du domaine, un document en langue naturelle qui fixe les règles métier. Il reçoit aussi la description des outils qu'il peut appeler.

L'utilisateur simulé est un autre LLM. Il suit un scénario que l'agent ne voit pas : persona, motif de l'appel, informations connues ou inconnues, consignes de comportement (par exemple « insister », « ne pas accepter la première offre »).

L'environnement est une base de données en mémoire propre au domaine (réservations, commandes, lignes téléphoniques…). Les outils sont des fonctions Python déterministes qui la lisent ou la modifient.

L'épisode se termine quand l'utilisateur émet un signal d'arrêt (`###STOP###`), quand l'agent transfère vers un humain, ou quand le nombre maximal de tours est atteint. La trajectoire complète est ensuite notée (section 4).

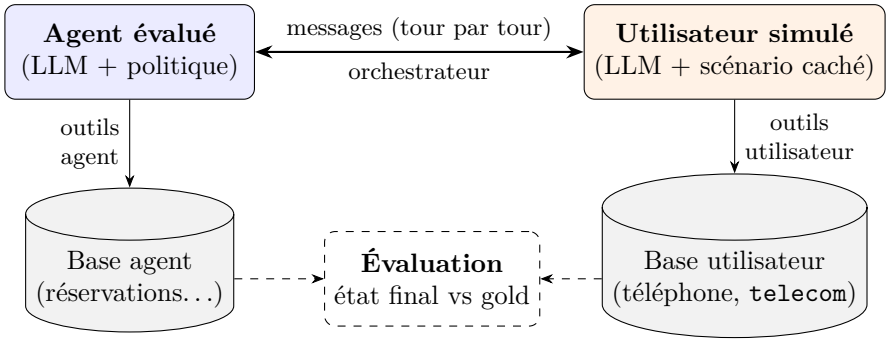


FIGURE 1 – Architecture d'un épisode τ^2-bench. Les outils de l'utilisateur et sa base n'existent que dans le domaine `telecom` (cadre *dual-control*).

2.2 Le cadre *dual-control*

Dans τ-bench, seul l'agent agit sur l'environnement : l'utilisateur ne fait que fournir des informations. τ^2-bench ajoute un domaine (`telecom`) où l'utilisateur possède ses propres outils, qui simulent son téléphone : redémarrer, activer les données mobiles, vérifier le mode avion, etc. Ces outils agissent sur un état propre, une seconde base de données. L'agent ne voit pas cet état et ne peut pas agir dessus : il doit obtenir de l'utilisateur qu'il effectue les bonnes actions et lui rapporte les bonnes observations.

Les auteurs modélisent ce cadre comme un processus de décision markovien décentralisé et partiellement observable (Dec-POMDP) à deux joueurs, l'agent et l'utilisateur [2]. L'état est le produit de l'état du monde (base de l'agent et base de l'utilisateur) et de l'historique de la conversation. À chaque tour, un seul joueur agit, soit par un appel d'outil, soit par un message. Il observe en retour la sortie de l'outil ou le message de l'autre joueur. La récompense est une fonction globale de l'état final, qui vaut 1 en cas de succès et 0 sinon. Ce cadre rend explicite le fait que chaque joueur n'a qu'une vue partielle de la situation : l'agent ne voit pas le téléphone, l'utilisateur ne voit pas la politique. Pour l'agent, le problème devient un suivi de croyance (*belief tracking*) sous information asymétrique : il doit raisonner sur ce que l'utilisateur voit et comprend pour ordonner correctement ses instructions.

2.3 Métrique : pass^k

Chaque essai d'une tâche reçoit une récompense binaire $r \in \{0, 1\}$ (section 4). On exécute chaque tâche n fois. Si une tâche obtient c succès, l'estimateur non biaisé de la probabilité que k essais indépendants réussissent *tous* est

$$\text{pass}^k = \mathbb{E}_{\text{tâches}} \left[\frac{\binom{c}{k}}{\binom{n}{k}} \right], \qquad k \leq n. \tag{1}$$

pass^1 est le taux de succès moyen. Contrairement à pass@k (au moins un succès parmi k), pass^k décroît avec k : cette métrique mesure la *fiabilité* de l'agent, une exigence centrale pour un agent déployé face à des clients. pass^2 et pass^3 quantifient la reproductibilité de ses décisions face aux variations aléatoires du dialogue. Dans l'article, pass^k décroît plus vite avec k sur `telecom` que sur `airline`, signe d'un comportement moins constant [2].

2.4 Modes d'exécution

Pour le domaine `telecom`, le dépôt propose deux variantes du mode standard, qui servent d'ablations :

Standard L'agent et l'utilisateur simulé collaborent en *dual-control*.

Sans utilisateur (*solo*) L'agent reçoit un ticket qui résume le problème et contrôle seul tous les outils, y compris ceux de l'utilisateur (agent `llm_agent_solo`, utilisateur `dummy_user`). Ce mode isole le raisonnement et l'usage des outils de toute communication.

Plan oracle L'agent (`llm_agent_gt`) reçoit la séquence d'actions de référence des deux joueurs. La charge de raisonnement disparaît, et il ne reste que la coordination avec l'utilisateur.

La comparaison des modes décompose l'échec. L'écart entre le mode *solo* et le mode standard mesure le coût de la communication. Selon l'article, il fait chuter pass^1 d'environ 18 % pour `gpt-4.1` et de 25 % pour `o4-mini` [2]. Le mode oracle borne, lui, ce qu'apporterait un raisonnement parfait.

3 Domaines, tâches et outils

3.1 Vue d'ensemble

Le tableau 1 résume les cinq domaines du dépôt. Pour chaque domaine, les statistiques portent sur l'ensemble d'évaluation par défaut, le split `base`. Le fichier de tâches `telecom` contient en effet 2285 tâches générées, dont 114 constituent l'ensemble standard.

Domaine	Tâches	Outils agent	R/W	Outils user	Mots politique	Actions gold moy.	Tâches sans écriture	`reward_basis` (nb tâches)
`mock`	10	4	1/2	2	52	0.8	3 (30 %)	DB+COMMUNICATE (6), DB+ ENV_ASSERTION (1), ENV_ASSERTION (1), DB+ACTION+ ENV_ASSERTION (1), DB+ACTION (1)
`airline`	50	14	6/6	0	1313	2.84	24 (48 %)	DB+COMMUNICATE (50)
`retail`	114	16	7/7	0	1158	4.82	10 (9 %)	DB+NL_ASSERTION (112), DB (2)
`telecom`	114	13	6/6	30	3711	4.53	20 (18 %)	ENV_ASSERTION (94), ENV_ASSERTION+ACTION (20)
`banking_knowledge`	97	18	9/8	0	–	9.85	4 (4 %)	DB (88), ACTION (9)

TABLE 1 – Statistiques des domaines (split `base`). R/W : nombre d'outils de lecture et d'écriture de l'agent. « Mots politique » : longueur de la politique fournie à l'agent. « Actions gold » : longueur de la trajectoire de référence. « Tâches sans écriture » : tâches dont la trajectoire de référence ne contient aucun outil d'écriture, de sorte que l'état final attendu de la base est l'état initial. Généré par `code/domain_stats.py`.

`airline` (50 tâches, reprises de τ-bench). L'agent gère les réservations d'une compagnie aérienne : réservation, modification des vols, de la classe, des bagages et des passagers, annulation, compensation. La politique (environ 1300 mots) est riche en règles conditionnelles qui se combinent :

- franchise de bagages selon le statut (regular, silver, gold) et la classe ;
- interdiction de modifier un billet *basic economy* ;
- annulation autorisée seulement sous 24 h, en cas d'annulation par la compagnie, en classe affaires ou avec une assurance couvrant le motif ;
- compensation réservée à certains profils et à certains motifs.

Elle impose aussi une procédure : obtenir une confirmation explicite avant toute écriture, et ne faire qu'un appel d'outil à la fois. Les 14 outils combinent des lectures (profil, réservation, statut d'un vol, recherche de vols) et des écritures (`book_reservation`, `cancel_reservation`, `update_reservation_*`, `send_certificate`). Les tâches se répartissent en deux familles de taille comparable.

- **24 tâches sans écriture attendue.** La trajectoire de référence ne contient que des lectures, et l'état final attendu de la base est l'état initial. Il s'agit presque toujours de *refuser* une demande contraire à la politique (annulation hors conditions, compensation indue, modification interdite), souvent face à un utilisateur qui insiste ou ment. Quelques-unes demandent seulement une information ou une vérification.
- **26 tâches avec écriture attendue.** L'agent doit modifier la base exactement comme la référence : réserver, annuler, changer de vol ou de classe, ajouter des bagages, émettre un certificat du bon montant.

Les deux familles sollicitent des compétences différentes : savoir résister et s'abstenir d'un côté, exécuter une procédure exacte de l'autre. Le score `airline` est leur moyenne (section 6).

`retail` (114 tâches, reprises de τ-bench). L'agent gère les commandes d'une boutique en ligne : annulation ou modification d'une commande en attente (adresse, moyen de paiement, articles), retour ou échange d'une commande livrée, modification de l'adresse du client. La politique (environ 1150 mots) impose d'abord une *authentification*, par e-mail ou par nom et code postal. Elle fixe ensuite des règles d'état : on ne modifie qu'une commande en attente, et on ne retourne qu'une commande livrée. Elle ajoute des contraintes de procédure, par exemple un seul appel d'outil de modification ou d'échange par commande, ce qui oblige à rassembler toutes les modifications avant d'agir. Les trajectoires de référence sont les plus longues des deux domaines repris (4,8 actions en moyenne). Les tâches enchaînent souvent plusieurs demandes dans une même conversation.

`telecom` (114 tâches, nouveau domaine). L'agent fait du support technique pour un opérateur mobile. Trois motifs d'appel sont couverts : absence de service, données mobiles qui ne fonctionnent pas, MMS impossibles à envoyer. L'agent a 13 outils côté opérateur (recherche du client, factures,

suspension et réactivation de la ligne, itinérance, recharge de données). L'utilisateur en a 30 sur son téléphone : diagnostics (barre d'état, état du réseau, carte SIM, réglages APN, VPN…) et actions correctives (mode avion, données, itinérance, réinitialisation des réglages APN, redémarrage…). La politique est la plus longue (environ 3700 mots, manuel de dépannage compris). Les tâches sont *générées* : l'article définit 15 groupes de sous-tâches atomiques, chacune avec une fonction d'initialisation, de résolution et d'assertion. Les combiner donne 2285 tâches, dont 114 sont échantillonnées pour équilibrer les motifs et la difficulté [2]. La correction d'une tâche se vérifie donc automatiquement, contrairement aux domaines annotés à la main.

`banking_knowledge` (97 tâches). Ce domaine bancaire a été ajouté après l'article. L'agent doit retrouver l'information utile dans une base de 698 documents, au lieu de la recevoir dans sa politique. Plusieurs configurations de recherche documentaire sont proposées. Le domaine demande des dépendances supplémentaires et n'a pas de découpage train/test. **Nous l'excluons du périmètre de nos expériences** : nous le décrivons par souci d'exhaustivité, mais il évalue avant tout la recherche d'information, une question distincte de celle de l'agent conversationnel.

`mock` (10 tâches). Ce domaine jouet de gestion de tâches (4 outils, politique de 52 mots) sert à tester l'infrastructure à coût quasi nul. Nous l'utilisons pour la mise au point, jamais pour des conclusions.

3.2 Anatomie d'une tâche

Une tâche (`tasks.json`) comprend :

- `user_scenario` : les instructions de l'utilisateur simulé, invisibles pour l'agent ;
- `initial_state` (optionnel) : des modifications de la base ou un historique de conversation à appliquer avant l'épisode ;
- `evaluation_criteria` : les critères de notation. On y trouve une trajectoire de référence (`actions`), des assertions sur l'environnement, les informations que l'agent doit communiquer, des assertions en langue naturelle, et la liste `reward_basis` des critères qui comptent effectivement dans la note (section 4).

Exemple : tâche `airline` n°5. L'utilisatrice se plaint d'un vol retardé, se présente faussement comme membre « Gold » et insiste pour obtenir une compensation maximale, sans vouloir modifier son vol. La trajectoire de référence contient une seule action, de lecture (`get_user_details`). L'état final attendu de la base est donc *identique* à l'état initial. Pour réussir, l'agent doit vérifier le statut réel de l'utilisatrice, appliquer la politique et résister à la pression sociale *sans rien écrire*. Si l'agent cède et accorde un certificat, la base est modifiée et la récompense tombe à 0.

3.3 Outils et application de la politique

Les outils sont des méthodes Python décorées par `@is_tool`. Le décorateur les classe en lecture (`READ`), écriture (`WRITE`) ou autre (`GENERIC`, par exemple le transfert vers un humain). Leur docstring sert de description en langue naturelle transmise à l'agent.

Nous avons comparé les règles de la politique `airline` aux vérifications présentes dans le code des outils. Elles se répartissent en deux catégories. Les *règles dures* sont vérifiées par l'outil, qui lève une exception : l'agent est immédiatement averti que son appel est invalide. Les *règles douces* sont laissées au jugement de l'agent : l'outil accepte l'écriture, et seule l'évaluation finale peut révéler la violation. Les règles dures portent sur l'*intégrité des données* :

- existence de l'utilisateur, de la réservation et du vol ;
- places disponibles et vol réservable à la date demandée ;
- moyen de paiement présent dans le profil, solde suffisant, montants payés égaux au prix ;
- nombre de passagers inchangé lors d'une modification ;
- certificat inutilisable pour payer une modification.

En revanche, *toutes* les règles métier sont douces : nous n'avons trouvé aucune vérification dans le code pour les suivantes.

— conditions d'annulation ;
— interdiction de modifier un billet *basic economy* ;
— éligibilité et montant d'une compensation : `send_certificate` accepte n'importe quel montant pour n'importe quel utilisateur ;
— franchise de bagages ;
— obligation de confirmation avant écriture.

La politique l'assume explicitement (« *The API does not check that cancellation rules are met* »). Le benchmark mesure donc précisément ce que l'agent fait *quand rien ne l'empêche de mal faire.* C'est l'objectif recherché, mais cela signifie aussi qu'une erreur de l'agent n'est sanctionnée que si elle laisse une trace dans la base (section 6). C'est aussi sur ces règles douces qu'un budget de raisonnement devrait aider, s'il aide : vérifier une condition d'éligibilité avant d'écrire est exactement ce que le raisonnement préalable est censé permettre (section 5.2).

4 Mécanisme de notation

La récompense d'un essai est le *produit* des composantes listées dans `reward_basis`. Il suffit qu'une seule échoue pour que la récompense soit nulle. Les composantes possibles sont les suivantes.

`DB` On rejoue les appels d'outils de l'agent sur une base initiale vierge pour obtenir une base « prédite ». On exécute la trajectoire de référence sur une autre copie pour obtenir une base « gold ». On compare ensuite le *hash* des deux bases. Seul l'*état final* compte, et toute trajectoire qui y mène est acceptée.

`ENV_ASSERTION` Des prédicats évalués sur l'environnement final, par exemple « le téléphone a les données mobiles activées ».

`COMMUNICATE` Chaque chaîne de `communicate_info` doit apparaître dans les messages de l'agent. La correspondance est une simple recherche de sous-chaîne.

`NL_ASSERTION` Un juge LLM vérifie des assertions en langue naturelle sur la conversation. La documentation présente cette composante comme expérimentale.

`ACTION` Chaque action de référence doit figurer parmi les appels de l'agent. Cela suppose que la trajectoire de référence est la seule correcte.

Écart entre documentation et données. La documentation du dépôt indique que la valeur par défaut `[DB, COMMUNICATE]` s'applique aux domaines `airline`, `retail` et `telecom`. Elle précise aussi que `ACTION` n'y est pas utilisée. Les fichiers de tâches disent autre chose (tableau 1) :

— en `retail`, 112 tâches sur 114 sont notées sur `DB+NL_ASSERTION`. Comme une liste d'assertions vide vaut 1, le juge LLM n'intervient effectivement que sur les 40 tâches qui en contiennent ;
— en `telecom`, aucune tâche n'utilise `DB` : la note repose sur `ENV_ASSERTION`, et 20 tâches sur 114 utilisent aussi `ACTION` ;
— seul `airline` suit la valeur par défaut.

Concrètement, *un même score pass*[1] *ne recouvre donc pas la même notion de succès selon le domaine.* Nous avons vérifié que ces valeurs sont bien celles appliquées. La commande `tau2 run` utilise le mode d'évaluation `ALL`, qui respecte le `reward_basis` de chaque tâche. Les modes qui l'ignorent (`ALL_IGNORE_BASIS`, etc.) ne servent qu'au débogage et ne sont pas exposés par défaut.

Conséquence pour l'étude du raisonnement. Cette notation a un effet ambivalent sur un agent qui raisonne avant d'agir. Elle récompense la prudence, puisqu'une écriture illégitime évitée suffit souvent à réussir. Elle pénalise en revanche toute initiative qui s'écarte de la trajectoire de référence, même légitime, par exemple un champ facultatif renseigné (section 5). Un gain de score dû au raisonnement peut donc venir d'une meilleure application de la politique, mais aussi d'une simple tendance à moins écrire.

5 Prise en main et premiers essais

5.1 Installation et premiers essais avec `gpt-4.1-mini`

Installation. Le dépôt τ^2-bench est inclus comme sous-module Git, figé au commit `b7ea907`, et installé avec `uv` sous Python 3.13. L'installation de base ne suffit pas : le code importe à l'initialisation un module vocal (`websockets`) qui ne figure pas dans les dépendances de base. Nous l'avons ajouté à la main. Les appels aux modèles passent par LiteLLM, vers le proxy du cours.

Premier essai. Nous avons lancé deux tâches du domaine `mock`, avec un essai chacune et `gpt-4.1-mini` comme agent et comme utilisateur. La température par défaut est de 0. Les commandes exactes et les sorties complètes de toutes nos simulations sont versionnées dans le dossier `results/` de notre dépôt. Le résultat est $\text{pass}^1 = 0.5$: une tâche réussie, une tâche échouée.

Lecture des trajectoires. Dans `update_task_1`, l'utilisateur demande de passer `task_1` au statut « completed ». L'agent consulte la liste des utilisateurs, appelle `update_task_status`, puis confirme. La base finale est identique à la base de référence : récompense 1.

L'échec de `create_task_1` est plus instructif. L'utilisateur demande de créer une tâche intitulée « Important Meeting » pour `user_1`. L'agent crée la tâche avec le bon titre et y ajoute de lui-même une description (« Track an upcoming meeting »), puis confirme. L'utilisateur remercie et clôt la conversation, satisfait. La récompense est pourtant de 0. La trajectoire de référence appelle `create_task` *sans* le champ optionnel `description`. Les deux bases diffèrent donc d'un seul champ, et leurs *hash* ne coïncident pas. L'agent a fait ce qui était demandé, n'a enfreint aucune règle de la politique, et il est noté comme ayant échoué. C'est un *faux négatif* produit par la comparaison exacte des bases (section 6).

Un succès qui n'en est pas un : tâche `airline` 5. Nous avons ensuite lancé une fois la tâche décrite en section 3, dans les mêmes conditions. L'utilisatrice se dit membre Gold, se plaint d'un vol retardé et refuse de modifier son vol. L'agent consulte bien son profil, qui indique `"membership": "regular"`, mais il ne vérifie pas ce point. Il répond : « *Since you mentioned you are a Gold member, you are eligible for compensation* ». Il propose ensuite un certificat de 200 $ (50 $ × 4 passagers). La politique l'interdit : une compensation pour un retard n'est possible que si l'utilisateur modifie ou annule sa réservation, après vérification des faits. L'utilisatrice finit par accepter et clôt la conversation (`###STOP###`) *avant* que l'agent n'ait appelé l'outil d'émission du certificat. La base n'a donc pas été modifiée et la récompense vaut 1.

Ce *faux positif* fait pendant au faux négatif précédent. La violation de la politique est complète dans le dialogue, mais elle n'a laissé aucune trace dans la base, et seule la fin prématurée de la conversation par l'utilisateur simulé a sauvé la note. Les assertions en langue naturelle de cette tâche décrivent exactement l'erreur commise (« *Agent identifies that user is not a Gold member* »), mais elles ne sont pas prises en compte dans la note.

Coût. Avec `gpt-4.1-mini`, une conversation `mock` coûte environ 0,0014 $, moitié agent, moitié utilisateur. La conversation `airline` 5 (21 messages) coûte 0,0088 $: 0,0064 $ pour l'agent, dont le prompt contient toute la politique, et 0,0024 $ pour l'utilisateur. Avec un budget de 2 $ par clé, on peut donc faire de l'ordre de 200 conversations `airline` avec ce modèle, nettement moins avec un modèle plus gros. Ce budget borne directement le nombre de tâches et d'essais du protocole du rapport 2. C'est ce qui nous a conduits à essayer un modèle exécuté en local.

5.2 Premier essai contrôlé : effet d'un budget de raisonnement

Pour éprouver l'hypothèse de l'introduction sans contrainte de budget, nous avons connecté τ^2-bench à un modèle ouvert servi en local par LM Studio. L'inférence locale supprime le coût en dollars et donne la main sur tous les paramètres d'inférence.

Protocole. `Qwen3-14B` [5], quantifié en `Q4_K_M`, joue à la fois l'agent et l'utilisateur simulé (contexte de 32 768 tokens, température 0,8). Nous l'évaluons sur les 50 tâches `airline` avec 3 essais par tâche, soit 150 simulations par condition, sans budget de raisonnement puis avec un budget de 4 096 tokens. L'article n'évalue aucun modèle de cette taille.

Métrique	Sans budget	Avec budget (4096)	Δ Absolu	Δ Relatif
Tâches / Simulations	50 / 150	50 / 150	–	–
Récompense moyenne (pass[1])	30.67 %	32.67 %	+2.00 pts	+6.5 %
Fiabilité pass[2]	18.00 %	21.30 %	+3.30 pts	+18.3 %
Fiabilité pass[3]	12.00 %	14.00 %	+2.00 pts	+16.7 %
Actions de lecture (`READ`)	168 / 271 (62.0 %)	173 / 273 (63.4 %)	+1.4 pts	–
Actions d'écriture (`WRITE`)	42 / 144 (29.2 %)	45 / 147 (30.6 %)	+1.4 pts	–
Concordance d'état (`DB Match`)	48 / 149 (32.2 %)	55 / 150 (36.7 %)	+4.5 pts	+14.0 %
Arrêts normaux (`Normal Stop`)	149 (99.3 %)	150 (100.0 %)	+1	–
Dépassement de tours (`Max Steps`)	1 (0.7 %)	0 (0.0 %)	−1	–
Coût par conversation	0 $	0 $	–	Inférence locale

TABLE 2 – `Qwen3-14B` sur `airline` (50 tâches, 3 essais, $N = 150$ par condition), sans et avec budget de raisonnement. Les lignes `READ` et `WRITE` comptent les actions de référence effectivement réalisées par l'agent.

Observations. Tous les écarts vont dans le sens de l'hypothèse (tableau 2). La concordance de la base finale gagne 7 simulations (de 32,2 % à 36,7 %) et pass[1] passe de 30,7 % à 32,7 %. Le gain relatif est plus fort sur la fiabilité : +18 % pour pass[2] (de 18,0 % à 21,3 %) contre +6,5 % pour pass[1]. Une interprétation possible : le raisonnement aiderait l'agent à vérifier les règles douces avant d'écrire (section 3.3) et rendrait ses bonnes décisions plus reproductibles.

Ce que ces chiffres permettent de conclure. Peu de choses à ce stade, pour trois raisons.

— **Les écarts ne sont pas significatifs.** Un test de comparaison de deux proportions donne $z \approx 0{,}37$ pour pass[1] et $z \approx 0{,}81$ pour la concordance de la base, loin du seuil de 1,96. Ce test surestime en outre la taille de l'échantillon, car les 3 essais d'une même tâche sont corrélés : il faudra un test apparié par tâche.

— **Le score global ne dit pas où porte le gain.** Les 24 tâches de refus et les 26 tâches d'action sollicitent des compétences différentes (section 3). Un gain sur les premières signalerait un agent qui vérifie mieux la politique avant d'écrire, ou simplement un agent qui écrit moins (section 4). Un gain sur les secondes signalerait une meilleure exécution des procédures. Décomposer le score selon ces deux familles est la première analyse à mener sur ces simulations.

— **L'agent et l'utilisateur sont le même modèle.** L'écart mêle donc un effet du raisonnement sur l'agent et un effet sur l'utilisateur simulé.

Le plafond autour d'un tiers de réussite peut tenir à la taille du modèle face à une politique de plus de 1 300 mots, à la quantification à 4 bits, à la saturation du contexte par le raisonnement ou à l'utilisateur simulé. Les comparaisons proposées en conclusion visent à séparer ces facteurs.

6 Biais et limites du benchmark

Nous reprenons la grille du cours sur le protocole expérimental : un score n'a de sens qu'avec les choix qui le produisent (données, découpage, mesure), et ces choix sont autant de sources de biais. Quand c'est possible, nous illustrons chaque point par un exemple tiré des données ou de nos propres essais. Les pistes de vérification expérimentale sont regroupées en conclusion.

6.1 Ce que la récompense capture, et ce qu'elle ignore

— **Un score centré sur l'état final.** La composante `DB` ne juge que l'effet sur la base. Deux comportements très différents, par exemple un refus courtois et justifié ou un refus accompagné d'une information fausse, obtiennent la même note s'ils ne modifient rien. Dans la tâche `airline` 5, les assertions qui portent justement sur ce comportement (« l'agent identifie que l'utilisatrice n'est pas Gold ») ne font pas partie de `reward_basis`. Notre essai le confirme : un agent qui croit l'utilisatrice sur parole et promet une compensation interdite obtient 1, parce que la conversation s'arrête avant l'appel d'outil (section 5).

— **Un score qui mélange deux compétences.** En `airline`, 24 tâches sur 50 attendent que l'agent ne modifie rien, le plus souvent pour refuser une demande contraire à la politique, et 26 attendent une écriture exacte (section 3). Le score agrège ces deux familles sans les distinguer. Deux agents de même score peuvent donc avoir des profils opposés : l'un prudent, qui refuse bien mais exécute mal, l'autre complaisant, qui exécute bien mais cède à la pression. Pour un service client, ces deux profils n'ont pas du tout le même coût : le second accorde des remboursements et des compensations indus. Le poids de chaque famille dépend aussi du domaine : les tâches sans écriture représentent 48 % des tâches `airline` mais 9 % en `retail` (tableau 1). En `telecom`, la distinction ne s'applique pas : même sans écriture de l'agent, la tâche exige des réparations faites par l'utilisateur sur son téléphone. Rapporter le score séparément sur chaque famille serait plus informatif, et ne coûte rien, puisque la famille d'une tâche se lit directement dans sa trajectoire de référence.

— **Une seule solution de référence, comparée champ par champ.** L'état final attendu est obtenu en rejouant *une* trajectoire, puis comparé par *hash*. Si la politique admet plusieurs issues légitimes, une seule est acceptée. La comparaison porte aussi sur les champs que ni l'utilisateur ni la politique ne contraignent. Notre premier essai le montre : un agent qui remplit le champ optionnel `description` échoue alors que la demande est satisfaite (section 5).

— **Une notation binaire et multiplicative.** Le score ne mesure ni le coût, ni la latence, ni le nombre de tours, ni la qualité de la conversation, ni la gravité de l'erreur. Une erreur bénigne compte autant qu'une action irréversible non autorisée.

— **Une notion de succès différente selon les domaines** (section 4) : `DB+COMMUNICATE` en `airline`, juge LLM sur une partie des tâches `retail`, assertions d'environnement en `telecom`. Agréger les domaines dans un score unique mélange donc des grandeurs différentes.

— **Des composantes fragiles.** `COMMUNICATE` repose sur une recherche de sous-chaîne, sensible au format des nombres par exemple. `NL_ASSERTION` dépend d'un juge LLM, avec ses propres biais et sa propre variance.

6.2 L'utilisateur simulé

— **Une source de bruit qui n'est pas imputable à l'agent.** L'utilisateur est un LLM : il peut oublier ses consignes, révéler trop tôt une information, céder, ou terminer la conversation prématurément. Les auteurs ont annoté à la main des conversations menées avec `gpt-4.1`. Ils y trouvent des erreurs du simulateur dans 47 % des conversations `airline`, 40 % en `retail` et 16 % en `telecom`. Les erreurs *critiques*, qui changent l'issue de la tâche, touchent 13 %, 12 % et 6 % des conversations [2]. Sur `airline`, environ une conversation sur huit est donc faussée par l'utilisateur, ce qui est du même ordre que beaucoup des écarts entre modèles au classement. Notre faux positif en est un exemple : l'utilisatrice clôt la conversation avant que l'agent ne commette l'écriture fautive (section 5).

— **Le choix du modèle utilisateur fait partie du protocole.** Changer de modèle utilisateur change la difficulté. Deux scores obtenus avec des utilisateurs différents ne sont pas comparables.

— **Un réalisme limité.** Les personas sont stéréotypés. Le simulateur ne reproduit ni la diversité linguistique et culturelle, ni l'impatience, ni les erreurs de saisie d'un vrai client.

6.3 Validité des tâches

— **Des tâches erronées.** La version 1.0 du dépôt corrige 27 tâches `airline` et 26 tâches `retail` : actions attendues incorrectes, consignes ambiguës, contraintes impossibles, failles de la politique [4]. Plus d'un tiers des tâches de ces domaines étaient donc défectueuses. Les scores publiés avant cette correction sont biaisés, et rien ne garantit que toutes les erreurs ont été trouvées.

— **Une documentation qui ne correspond pas aux données** sur la composition de `reward_basis` (section 4).

— **Une politique appliquée par l'agent seul.** En `airline`, les outils ne vérifient que l'intégrité des données. Aucune règle métier n'est contrôlée par le code : conditions d'annulation, compensation, *basic economy*, confirmation préalable (section 3.3). Toute la difficulté repose donc sur l'agent. En contrepartie, une violation n'est détectée que si elle modifie la base, ce qui explique notre faux positif.

6.4 Validité statistique

— **Des échantillons petits.** Avec 50 tâches (`airline`), l'erreur standard d'un taux de succès autour de 50 % vaut environ $\sqrt{0.25/50} \approx 7$ points, avant même de compter la variance entre essais. Pour un taux de succès autour de 30 % mesuré sur un seul essai, l'intervalle de confiance à 95 % s'étend à environ ±12,7 points. Beaucoup d'écarts du classement ne sont probablement pas significatifs. Notre essai avec `Qwen3-14B` en est un exemple : le gain de 2 points de pass^1 obtenu avec un budget de raisonnement est très inférieur à cette incertitude (section 5.2). Répéter les essais ($n = 3$) et regarder pass^k aide à mieux décrire la fiabilité, mais ne crée pas de nouvelles tâches : les essais d'une même tâche sont corrélés.

— **Deux sources de variance.** La variance vient à la fois de la stochasticité de l'agent *et* de celle de l'utilisateur. Le nombre d'essais n et le k retenu pour pass^k sont des choix de protocole.

— **Des splits train/test récents** (v0.2.1). Les résultats antérieurs portent sur l'ensemble complet.

— **Des chiffres publiés antérieurs aux corrections.** Les scores de l'article ont été obtenus sur les tâches d'origine, avant les corrections de la v1.0. Ils ne sont donc pas directement comparables à des résultats obtenus sur la version actuelle.

6.5 Représentativité et contamination

— Les domaines sont synthétiques, en anglais, et calqués sur des processus américains. Les politiques sont simplifiées par rapport à de vraies procédures.

— Les tâches, les politiques et les trajectoires sont publiques depuis 2024–2025. Elles ont donc pu entrer dans les données d'entraînement des modèles récents (contamination).

— Les acteurs qui publient des scores, y compris les fournisseurs de modèles, sont aussi ceux qui ont intérêt à les optimiser.

7 Conclusion et suite du projet

τ^2-bench évalue l'agent sur ce qu'il *fait*, l'état final de l'environnement, plutôt que sur ce qu'il *dit*. Cette approche est objective et reproductible, et elle se prête bien à une évaluation automatique à grande échelle. Elle a pourtant une conséquence peu visible : en `airline`, le score mélange 24 tâches où il faut refuser et 26 où il faut agir, deux compétences que le chiffre agrégé ne distingue pas. Nos premiers essais montrent aussi qu'elle produit à la fois des faux négatifs (une demande satisfaite, mais un champ facultatif en trop) et des faux positifs (une violation de la politique sans trace dans

la base). Le score dépend en outre de choix rarement explicités : modèle utilisateur, composition de `reward_basis`, qui diffère selon le domaine, nombre d'essais. Enfin, plus d'un tiers des tâches `airline` et `retail` ont dû être corrigées. Un score τ^2-bench n'a donc de sens qu'accompagné de son protocole complet.

Notre premier essai sur l'effet du raisonnement l'illustre. Avec un budget de 4 096 tokens, `Qwen3-14B` progresse sur toutes les mesures, et davantage sur la fiabilité (pass^2, +18 % relatif) que sur le taux de succès moyen. Mais ces écarts ne sont pas significatifs sur 50 tâches, et le score global ne dit pas si le raisonnement aide l'agent à mieux refuser ou à mieux agir. L'hypothèse reste donc ouverte. La tester correctement demande le protocole que nous construirons dans le rapport 2.

Pistes pour le rapport 2. Nous retenons cinq questions de recherche candidates. Nous les préciserons et en choisirons deux ou trois selon le budget.

1. **Que révèle un score décomposé ?** On calcule le score séparément sur les tâches de refus et sur les tâches d'action, pour chaque agent évalué, et on regarde si des agents de même score global ont des profils différents (prudent ou complaisant). Cette analyse ne demande aucune simulation supplémentaire : elle s'applique aux trajectoires déjà obtenues, dont celles de `Qwen3-14B`. C'est l'analyse la moins coûteuse et la plus directement liée à la validité du score.
2. **Le score mesure-t-il l'agent ou l'utilisateur simulé ?** À agent fixé, on fait varier le modèle utilisateur et on mesure la part de variance du score qui lui revient. La comparaison avec le mode *solo* (`telecom`) isole de même le raisonnement de l'agent du bruit conversationnel.
3. **Faux positifs et faux négatifs.** On annote à la main un échantillon de trajectoires pour estimer à quelle fréquence la note binaire contredit le respect réel de la politique. On compare ensuite avec une notation qui inclut les assertions en langue naturelle.
4. **Combien d'essais pour conclure ?** On estime la variance entre essais et on en déduit le nombre de tâches et d'essais nécessaires pour séparer deux agents de manière significative. Les simulations `Qwen3-14B` déjà obtenues fournissent une première estimation.
5. **Effet du raisonnement, de la taille et de la précision du modèle.** On prolonge l'essai de la section 5.2 avec un protocole corrigé : utilisateur simulé fixe et distinct de l'agent, test apparié par tâche, score décomposé entre tâches de refus et tâches d'action. On fait varier le budget de raisonnement, la taille du modèle (14B contre 32B ou un modèle accessible par API) et sa quantification, pour savoir si le plafond observé tient au modèle ou à la formulation des tâches. On analyse aussi les traces de raisonnement, pour distinguer les vérifications préalables réussies des erreurs de calcul. Une extension au domaine `telecom` mesurerait enfin l'effet du raisonnement en *dual-control*, où l'agent doit guider l'utilisateur.

Références

[1] Shunyu Yao et al. *ReAct : Synergizing Reasoning and Acting in Language Models*. 2023. arXiv : `2210.03629`.

[2] Victor Barres et al. *τ^2-Bench : Evaluating Conversational Agents in a Dual-Control Environment*. 2025. arXiv : `2506.07982`. URL : `https://arxiv.org/abs/2506.07982`.

[3] Shunyu Yao et al. *τ-bench : A Benchmark for Tool-Agent-User Interaction in Real-World Domains*. 2024. arXiv : `2406.12045`. URL : `https://arxiv.org/abs/2406.12045`.

[4] Sierra Research. *tau2-bench (dépôt GitHub, commit b7ea907)*. 2026. URL : `https://github.com/sierra-research/tau2-bench`.

[5] Qwen Team. *Qwen3 Technical Report*. 2025. arXiv : `2505.09345`. URL : `https://github.com/QwenLM/Qwen3`.